Celo Pro League
Findings & Analysis Report

2024-08-12

Table of contents

- Overview
  - About C4
  - Wardens
- Summary
- Scope
- Severity Criteria
- Informational Findings (1)
  - I-1 Inconsistent vote accounting between strategies after upgrade to changeStrategy()
- Disclosures

Overview

About C4

Code4rena (C4) is an open organization consisting of security researchers, auditors, developers, and individuals with domain expertise in smart contracts.

A C4 Pro League Audit is an event where elite tier Code4rena contributors, commonly referred to as wardens, reviews, audits and analyzes smart contract logic in exchange for a bounty provided by sponsoring projects.

During the Pro League audit outlined in this document, C4 conducted an analysis of the Celo smart contract system written in Solidity. The audit took place between June 25 - July 29, 2024.

Wardens

2 Wardens contributed to Celo:

1. MiloTruck
2. HolladieWaldfee

Final report assembled by bytes032 and Sentinel

Summary

The C4 Pro League analysis yielded no HIGH or MEDIUM severity vulnerabilities.

Additionally, C4 Pro League analysis included 1 finding with an informational rating.

Scope

The source code was delivered to Code4rena in a private Git repository.

Severity Criteria

C4 assesses the severity of disclosed vulnerabilities based on three primary risk categories: high, medium, and low/non-critical.

High-level considerations for vulnerabilities span the following key areas when conducting assessments:

- Malicious Input Handling
- Escalation of privileges
- Arithmetic
- Gas use

For more information regarding the severity criteria referenced throughout the submission review process, please refer to the documentation provided on the C4 website, specifically our section on Severity Categorization.

Informational Findings (1)

[I-1] Inconsistent vote accounting between strategies after upgrade to changeStrategy()

Context:

- Manager.sol#L369-L373

Description:

PR #204 introduces a change to `changeStrategy()`, where the amount of stCelo transferred from the old strategy to the new strategy now includes the caller's locked vote balance:

```
- uint256 stCeloAmount = stakedCelo.balanceOf(msg.sender);
+ uint256 stCeloAmount = stakedCelo.balanceOf(msg.sender) +
+     stakedCelo.lockedVoteBalanceOf(msg.sender);
  if (stCeloAmount != 0) {
```

```
        _transfer(strategies[msg.sender], newStrategy, stCeloAmount);
    }
```

However, if a user calls `changeStrategy()` before the upgrade, their accounting between strategies becomes inconsistent after the upgrade to the `Manager` contract is performed.

Consider the following scenario:

- Assume a user has a portion of their stCelo locked in votes in strategy A.
- A user calls `changeStrategy()` to switch to strategy B before the upgrade.
- The upgrade is performed.
- The user attempts to call `changeStrategy()` to switch to strategy C.

When the user calls `changeStrategy()` post-upgrade, `_transfer()` is called to move `stCeloAmount` from strategy B to strategy C. However, a portion of `stCeloAmount` is still in strategy A as locked votes, so the transfer is not possible and will revert.

Recommendation:

The team has implemented a follow-up PR that addresses inconsistent vote accounting.

Celo:

It is not a problem that accounting become non-consistent after shipping a fix, because we could fix it with the balance with a follow up governance proposal calling `updateGroupStCelo()` in **PR #205**.

C4 Pro League:

Acknowledged as a non-issue.

While the balance discrepancy for users caused by the previous bug **has been addressed**, the team should consider the case where a user is intentionally malicious and tries to mess up the accounting. For example, they could front-run or back-run the upgrade and/or the subsequent governance proposal.

Disclosures

C4 is an open organization governed by participants in the community.

C4 audits incentivize the discovery of exploits, vulnerabilities, and bugs in smart contracts. Security researchers are rewarded at an increasing rate for finding higher-risk issues. Audit submissions are judged by a knowledgeable security researcher and solidity developer and disclosed to sponsoring developers. C4 does not conduct formal verification regarding the provided code but instead provides final verification.

C4 does not provide any guarantee or warranty regarding the security of this project. All smart contract software should be used at the sole risk and responsibility of users.

Top

An open organization | Twitter | Discord | GitHub | Blog | Newsletter | Media kit | Careers | code4rena.eth